

卫星仿真项目本地测试指南

环境安装、分层模型验证、Agent 闭环、证据门禁和发布重新冻结

文档版本	V2.0
项目版本	/ satellite-simulation-platform 0.3.3
编制日期	2026-07-11
文档性质	工程建模与使用说明
编制依据	最终源码包、正式 140×5 Golden Set、～ 硬化报告和 发布收口报告

适用于 最终完整源码包、拆分制品及 0.3.3 wheel

目录

1. 指南范围与测试基线
2. 测试前准备
3. 安装源码包或 wheel
4. 环境资产配置
5. L0 环境诊断
6. L1 静态和单元测试
7. L2 分层模型门禁
8. L3 Agent 规划与脚本导出
9. L4 实际代表场景
10. L5 Golden Set
11. L6 本地 API
12. L7 发布收口检查
13. Run Bundle 与结果判读
14. 常见问题排查
15. 本地测试记录模板
16. 附录：推荐命令清单
17. 发布硬化与制品验证

注：本页目录为可更新的 Word 目录域，并保留当前章节索引作为缓存结果；内容变更后可在 Word 中按 F9 更新。

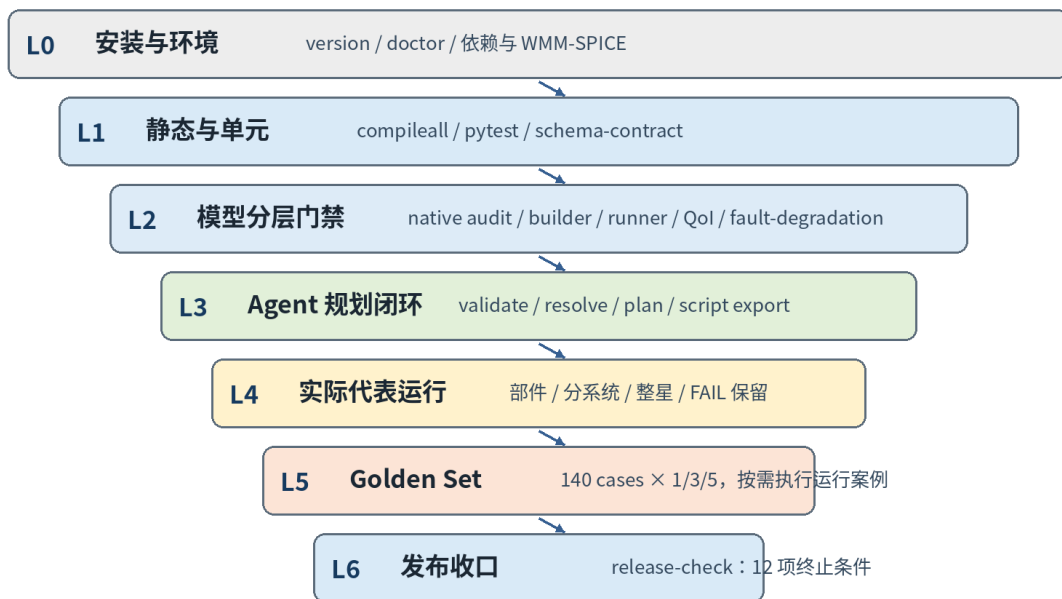
1. 指南范围与测试基线

本指南面向 最终完整包、拆分制品和 0.3.3 wheel，用于在本地确认环境、分层物理模型、故障/退化、Agent 规划、实际 Runner、Run Bundle、四态验证、effect-QoI 证据和发布门禁。命令以 Linux/WSL 为主，同时给出 Windows PowerShell 的关键差异。

正式发布基线	期望值
包版本	0.3.3
Python 要求	>= 3.10
核心依赖	bsk 2.11.0、sgp4 2.25、colorama 0.4.6、numpy 2.3.5、pydantic 2.13.4、PyYAML 6.0.3、jsonschema 4.26.0
环境 doctor	11 PASS / 0 FAIL
pytest 正式基线	121 passed（可按隔离进程分组复现）
Golden Set	140/140；repeat_count=5；700 次规范化/规划检查稳定；首轮执行 7 个标记 Runner 案例
发布终止条件	12/12 PASS
代表实际场景	6/6 结果符合预期：5 个 PASS，1 个预期 FAIL

本地结果原则：上述为发布基线，不应在未执行时预先记录为本地通过。机器、Python 版本、资产路径和可用时间不同，完整套件可能耗时较长。

本地测试分级与推荐执行顺序



推荐：日常修改执行 L0-L3；模型/事件修改执行至 L4；发布候选执行 L0-L6。

图 1 推荐本地测试等级

2. 测试前准备

2.1 解压与目录检查

```

unzip src_bsk_agent_v33_release_full.zip
cd src_bsk_agent_v33_release_full

# 至少应看到
README.md pyproject.toml requirements.txt
src/ configs/ examples/ tests/ scripts/ docs/ reports/ third_party/
  
```

Windows PowerShell:

```

Expand-Archive .\src_bsk_agent_v33_release_full.zip -DestinationPath .
Set-Location .\src_bsk_agent_v33_release_full
  
```

2.2 创建隔离环境

Linux / WSL:

```

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
  
```

Windows PowerShell:

```

py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
  
```

Python 选择：项目声明 Python ≥ 3.10 。若当前 Python 没有可用的 bsk 2.11.0 wheel，应切换到有 wheel 的 Python 环境；不要通过忽略 Basilisk 导入错误继续执行物理测试。

3. 安装源码包或 wheel

3.1 推荐：完整源码包

```
python -m pip install -r requirements.txt
python -m pip install -e ".[api,dev]"
```

如果 bsk 依赖解析失败，可拆分安装：

```
python -m pip install bsk==2.11.0 --no-deps
python -m pip install colorama==0.4.6 sgp4==2.25 numpy==2.3.5 \
    pydantic==2.13.4 PyYAML==6.0.3 jsonschema==4.26.0
python -m pip install "fastapi>=0.115,<1" "uvicorn>=0.30,<1" \
    "httpx>=0.27,<1" "pytest>=8,<10" "build>=1.2,<2"
python -m pip install -e .
```

3.2 Wheel 安装

```
python -m pip install "satellite_simulation_platform-0.3.3-py3-none-any.whl[api]"
python -m pip install "pytest>=8,<10" "httpx>=0.27,<1"
```

wheel 边界：轻量 wheel 不包含完整 SPICE 内核和源码测试脚本。需要严格环境测试、全部 pytest 或模型验证脚本时，使用完整源码包。

3.3 验证安装身份

```
sat-agent version
python -c "import Basilisk, sat_sim; print(Basilisk.__version__, sat_sim.__version__)"
```

期望：Basilisk 2.11.0，sat_sim 0.3.3，release manifest 为。源码 editable 安装时，若环境中残留旧 distribution metadata，package.version 可先显示 WARN；最终 wheel 的隔离安装必须为 PASS。

4. 环境资产配置

完整源码包默认包含：

```
third_party/wmm/WMM2025.COF
third_party/spice/<kernel files>
```

使用 wheel 或外部资产时，可配置：

```
# Linux / WSL
export SAT_SIM_ASSET_ROOT=/path/to/assets
```

```
# 或分别指定
export SAT_SIM_WMM_PATH=/path/to/WMM2025.COF
export SAT_SIM_SPICE_PATH=/path/to/spice_dir

# PowerShell
$env:SAT_SIM_ASSET_ROOT="D:\sat_assets"
$env:SAT_SIM_WMM_PATH="D:\sat_assets\wmm\WMM2025.COF"
$env:SAT_SIM_SPICE_PATH="D:\sat_assets\spice"
```

检查模式	命令	缺少资产时
普通开发	sat-agent doctor --require-api	WMM/SPICE 可为非阻断提醒，取决于场景。
严格高保真	sat-agent doctor --require-api --strict-assets	缺少 WMM/SPICE 直接 FAIL。

5. LO 环境诊断

```
sat-agent doctor --require-api --output reports/local_doctor.json

# 发布候选或高保真环境
sat-agent doctor --require-api --strict-assets \
--output reports/local_doctor_strict.json
```

正式源码包的期望检查项：

检查	期望
python.version	PASS; Python >= 3.10。
dependencies.core	固定依赖均可导入且版本匹配。
dependencies.api	FastAPI、Uvicorn、HTTPX 可用。
package.version	模块、distribution 和 release manifest 均为 0.3.3。
registry.capabilities	42 contracts、26 active、0 error、0 candidate warning；15 项 effect 已最终分类。
package.schemas	12 份发布 Schema 全部存在。
package.golden_set	140 cases。
filesystem.runs / artifacts	目录可写。
assets.environment	严格模式下 WMM/SPICE 全部解析。
workflow.smoke	Form → TaskSpec → Plan → Compile 通过。

Effect 证据判读：不再保留 candidate 状态。13 项为已验证的注册 source-native 实现效果，1 项为显式 project proxy，1 项为 out_of_scope；不得扩大为硬件标定、飞行相关或认证声明。

6. L1 静态和单元测试

6.1 编译检查

```
# editable 安装后
python -m compileall -q src scripts tests

# 未安装项目时 (Linux/WSL)
PYTHONPATH=.:src python -m compileall -q src scripts tests
```

期望：命令返回 0，无 SyntaxError。

6.2 快速 Agent/发布专项测试

```
PYTHONPATH=.:src python -m pytest -q \
  tests/test_agent_v20_v21.py \
  tests/test_agent_v22_v23.py \
  tests/test_agent_v24.py \
  tests/test_agent_v25_v26.py \
  tests/test_agent_v27.py \
  tests/test_agent_v28.py \
  tests/test_release_hardening_v29.py \
  tests/test_release_hardening_v30.py \
  tests/test_release_hardening_v31.py \
  tests/test_release_hardening_v32.py \
  tests/test_release_refreeze_v33.py
```

6.3 全量 pytest

```
PYTHONPATH=.:src python -m pytest -q
```

正式 基线为 121 passed。若单进程执行受 Basilisk/SWIG 进程级生命周期影响，应按 Agent 测试与模型/发布测试分组在全新进程中执行，并保留两组报告；不得把超时写成通过。

导入 scripts 失败：测试环境应同时包含项目根目录和 src：Linux/WSL 使用 PYTHONPATH=.:src；PowerShell 使用 \$env:PYTHONPATH="\$PWD;\$PWD\src"。

7. L2 分层模型门禁

以下脚本检查层级所有权、原生映射、builder/runner、环境资产、QoI、运行时事件和整星装配。日常修改可选择受影响领域；发布候选应执行正式闭环集合。

领域	推荐命令	主要验证
层级和 Schema	python scripts/check_layer_schema_contract.py	依赖方向、Config/Spec 位置、ghost 参数和越层调用。
原生能力审计	python scripts/check_native_capability_audit_v2.py	native/proxy/out_of_scope 分类和公开参数消费。
参数证据	python scripts/check_parameter_provenance_calibration_1.py	参数 registry、证据等级、Claim ceiling。
环境/ADCS	python scripts/check_env_data_asset_adcs_sensor_native.py python scripts/check_adcs_actuator_native_2.py	WMM/SPICE、传感器和执行机构原生映射。
分系统 builder/runner	python scripts/check_subsystem_basilisk_builders.py python scripts/check_subsystem_basilisk_runners.py	部件导入、消息链和可执行场景。
故障/退化	python scripts/check_subsystem_fault_degradation_coverage.py python scripts/check_runtime_injection_closure_2.py	场景覆盖、应用/恢复、无静默 unsupported。
QoI	python scripts/check_subsystem_basilisk_qoi_matrix.py python scripts/check_whole_spacecraft_basilisk_qoi.py	输出存在、方向性和任务指标。

领域	推荐命令	主要验证
整星	python scripts/check_whole_spacecraft_structure.py python scripts/check_wholesc_native_closure_1.py	中心总线、分系统 owned 装配和整星能力闭环。
文档/发布	python scripts/check_validation_doc_closure_2.py	报告、边界和验证文档一致性。
代码硬化	python scripts/check_runtime_static_public_api_v29.py / python scripts/check_duplicate_definition_dead_code_v30.py / python scripts/check_silent_exception_runtime_diagnostics_v31.py	未定义名称、损坏导出、重复定义和静默异常门禁。
Effect 证据	python scripts/check_effect_qoi_evidence_closure_v32.py	15 项 effect 最终分类、配对运行、QoI 和 Claim 边界。

推荐统一前缀：

```
# Linux / WSL
PYTHONPATH=.:src python scripts/<script>.py

# PowerShell
$env:PYTHONPATH="$PWD;$PWD\src"
python scripts\<script>.py
```

7.1 聚合套件

```
PYTHONPATH=.:src python scripts/run_basilisk_validation_suite.py --profile fast
```

耗时边界： full profile 含历史长时间检查，可能超过普通交互窗口。正式发布应保存聚合报告；日常开发优先 fast profile 和受影响的 focused gates。

8. L3 Agent 规划与脚本导出

```
sat-agent validate examples/agent_release_v33/component_nominal.yaml
```

```
sat-agent resolve examples/agent_release_v33/component_nominal.yaml
sat-agent plan examples/agent_release_v33/whole_spacecraft_combined_nominal.yaml
sat-agent export-script examples/agent_release_v33/component_nominal.yaml \
  --output generated/component_nominal.py
```

自然语言模板基线：

```
sat-agent parse "创建电池部件正常仿真，运行 300 秒" \
  --backend template --output-dir generated/parse_smoke
```

检查点	通过条件
TaskSpec	schema_version=1.0.0；字段 provenance 可追踪。
ResolvedSpec	capability、level、owner、依赖、effect/output binding 明确。
ExecutionPlan	DAG 通过校验，Plan Hash 稳定，无 REQUEST_LEVEL_CONFLICT。
脚本导出	只调用登记的公开 Adapter，不含模型自由 Python。

9. L4 实际代表场景

建议先运行正常场景，再执行故障/退化和预期 FAIL 场景。

```
mkdir -p runs/local_v33

sat-agent run examples/agent_release_v33/component_nominal.yaml \
  --output-root runs/local_v33

sat-agent run examples/agent_release_v33/subsystem_nominal.yaml \
  --output-root runs/local_v33

sat-agent run examples/agent_release_v33/whole_spacecraft_combined_nominal.yaml \
  --output-root runs/local_v33
```

示例	正式预期	进程返回
component_nominal.yaml	SUCCEEDED + PASS	0
subsystem_nominal.yaml	SUCCEEDED + PASS	0
whole_spacecraft_combined_nominal.yaml	SUCCEEDED + PASS	0
component_fault.yaml	SUCCEEDED + PASS; Delivery/Effect PASS	0

示例	正式预期	进程返回
component_degradation.yaml	SUCCEEDED + PASS; Delivery/Effect PASS	0
mission_fail_preserved.yaml	SUCCEEDED + FAIL	1（预期）

返回码解释：sat-agent run 仅在执行成功且 ValidationOutcome=PASS 时返回 0。返回 1 可能是任务 FAIL/INCONCLUSIVE，不等同于 Python 异常；返回 2 表示命令或执行异常。

也可以一次执行正式代表套件：

```
sat-agent release-check \  
  --golden-report reports/agent_golden_v33/golden_eval_report.json \  
  --output-dir reports/local_release_v33 \  
  --source-root . \  
  --strict-assets
```

10. L5 Golden Set

用途	命令	说明
快速抽样	sat-agent eval --repeat-count 1 --max-cases 20 --output-dir reports/local_golden_quick	验证 Manifest、规则和规划基本可用。
全量单次	sat-agent eval --repeat-count 1 --output-dir reports/local_golden_once	140 案例，不执行标记运行案例。
正式重复	sat-agent eval --repeat-count 5 --execute-marked --output-dir reports/local_golden_v33	140×5；每个案例重复 5 次，标记的 7 个实际 Runner 案例仅在首轮执行。

正式基线指标：

指标	基线
case_pass_rate	1.0
false_accept_rate / false_reject_rate	0.0 / 0.0
level_misroute_rate / effect_owner_error_rate	0.0 / 0.0
same_input_same_taskspec_rate / same_input_same_plan_rate	1.0 / 1.0
pass@1 / pass^3 / pass^5	1.0 / 1.0 / 1.0

指标	基线
runtime injection delivery success	1.0
runtime physical effect verified	1.0; 13 项注册实现效果具备直接 QoI 证据; proxy/out_of_scope 边界保持显式
Run Bundle integrity	1.0

11. L6 本地 API

```
sat-agent serve --host 127.0.0.1 --port 8000 \
--runs-root runs/api --artifacts-root .sat_sim_api
```

另开终端：

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/release
curl http://127.0.0.1:8000/capabilities

sat-agent openapi --output reports/local_openapi.json
```

接口组	路径
健康与发布	GET /health、/health/details、/release
能力与策略	GET /capabilities、/schema/taskspec、/policy/tools
任务	POST /tasks/parse、/validate、/resolve、/plan
运行	POST /runs、/runs/{id}/execute、/cancel
查询	GET /runs/{id}、/artifacts、/report

API 边界：本地 API 同步执行，无鉴权、TLS、多租户和分布式队列。不要直接暴露到不可信网络。

12. L7 发布收口检查

```
sat-agent release-check \
--golden-report reports/agent_golden_v33/golden_eval_report.json \
--output-dir reports/local_release_v33 \
--source-root . \
--strict-assets
```

期望输出目录：

```
reports/local_release_v33/
```

```

|—— environment_doctor.json
|—— representative_suite.json
|—— release_closure_report.json

```

12 项终止条件必须全部 PASS：Schema、Golden Set、unsupported 拒绝、Claim 门禁、确定性、三层运行、四类场景、注入真值、FAIL 保留、CLI/API、Run 证据和文档交接。

13. Run Bundle 与结果判读

```

sat-agent inspect runs/local_v33/<run_id>
sat-agent report runs/local_v33/<run_id>

```

文件	重点查看
run_record.json	运行状态、当前 attempt、Plan Hash、时间和 supersedes。
runtime/execution_trace.json	DAG 节点、分支和终态。
results/summary.json	模型摘要和关键输出。
validation/validation_outcome.json	PASS/FAIL/INCONCLUSIVE/NOT_EVALUATED、断言和注入证据。
validation/claim_report.json	允许/禁止声明、limitations、reason codes。
bundle_manifest.json / SEALED.json	文件 Hash 和不可变封存。

现象	正确解释
SUCCEEDED + PASS	仿真和任务验证均成功。
SUCCEEDED + FAIL	模型运行成功，但任务指标明确失败；不得重写成 PASS。
SUCCEEDED + INCONCLUSIVE	必要 QoI 或因果证据不足；可补模型/证据后创建新 Run。
FAILED/CANCELLED + NOT_EVALUATED	执行未完成，不评价任务。
Delivery PASS + Effect PASS	事件已投递，且 effect-specific QoI/直接状态证据满足；声明仍受 implementation 和参数证据上限约束。

14. 常见问题排查

问题/Reason Code	原因	处理
ModuleNotFoundError: Basilisk	bsk 未安装或 Python 无匹配 wheel。	在虚拟环境安装 bsk==2.11.0；必要时 --no-deps；换用可支持的 Python。

问题/Reason Code	原因	处理
doctor strict-assets FAIL	WMM/SPICE 路径缺失。	使用完整源码包或设置 SAT_SIM_ASSET_ROOT/WMM_PATH/SPICE_PATH。
REQUEST_LEVEL_CONFLICT	用户要求层级与最终 capability 不一致。	修改 capability 或 TaskSpec level，不要绕过门禁。
REQUIRED_QOI_MISSING	能力不能产生请求的 Qoi。	选择具备该输出的能力，或先扩展模型和 Contract。
UNSUPPORTED_EFFECT / owner mismatch	effect 不在依赖闭包或 target 错误。	检查 Capability Contract、owner 和层级。
MODEL_EXTENSION_REQUIRED / PROXY_EFFECT_NOT_ALLOWED	effect 为 out_of_scope，或代理效果未获得显式许可。	扩展模型；或在明确接受代理边界时设置 assurance.allow_proxy=true。
run 返回 1 但日志无异常	任务 FAIL 或 INCONCLUSIVE。	使用 sat-agent report 查看 ValidationOutcome。
pytest 无法导入 scripts	PYTHONPATH 只有 src。	设置 PYTHONPATH=.:src。
full suite 超时	历史/实际仿真较长。	先执行 fast profile 和 focused gates；发布时使用可观察长任务保存报告。
Run Bundle integrity FAIL	封存后文件被修改。	保留旧 Run，新建 superseding Run；不要改写旧制品。

15. 本地测试记录模板

记录项	填写内容
测试日期	
操作系统 / CPU / 内存	
Python / 虚拟环境	
Basilisk / sat_sim 版本	
源码包或 wheel SHA-256	
WMM/SPICE 路径和 Hash	
doctor 结果	PASS/WARN/FAIL:
pytest 结果	passed/failed/duration:

记录项	填写内容
Focused gates	执行脚本及报告：
代表场景	各场景 ValidationOutcome：
Golden Set	case/repeat/metrics：
Release check	12 项结果：
未通过项与日志路径	
结论	可用于开发 / 可用于发布候选 / 不通过

报告保存：建议将所有 JSON 报告、命令行日志、环境版本、数据资产 Hash 和运行制品归档到一个带日期的目录，不要只保存终端截图。

16. 附录：推荐命令清单

```
# 1. 安装
python -m pip install -r requirements.txt
python -m pip install -e "[api,dev]"

# 2. 环境
sat-agent version
sat-agent doctor --require-api --strict-assets --output reports/local_doctor.json

# 3. 静态与测试
PYTHONPATH=.:src python -m compileall -q src scripts tests
PYTHONPATH=.:src python -m pytest -q

# 4. 关键模型门禁
PYTHONPATH=.:src python scripts/check_layer_schema_contract.py
PYTHONPATH=.:src python scripts/check_native_capability_audit_v2.py
PYTHONPATH=.:src python scripts/check_runtime_injection_closure_2.py
PYTHONPATH=.:src python scripts/check_wholesc_native_closure_1.py

# 5. Agent 与实际运行
sat-agent validate examples/agent_release_v33/component_nominal.yaml
sat-agent plan examples/agent_release_v33/whole_spacecraft_combined_nominal.yaml
sat-agent run examples/agent_release_v33/component_nominal.yaml --output-root runs/local

# 6. 评测和发布
sat-agent eval --repeat-count 5 --execute-marked --output-dir reports/local_golden_v33
```

```
sat-agent release-check --golden-report reports/local_golden_v33/golden_eval_report.json \
--output-dir reports/local_release_v33 --source-root . --strict-assets
```

17. 发布硬化与制品验证

在 ~ 的代码硬化和 effect-QoI 收口基础上重新冻结发布。下列检查不替代分层物理门禁，而是作为正式制品发布的补充门禁。

检查	命令	通过条件
静态正确性	ruff check src tests scripts --select F821,F822,F811,F841	全部为 0
重复定义	python scripts/check_no_duplicate_module_definitions.py	重复顶层定义为 0
异常策略	python scripts/check_exception_policy_v31.py	静默 except: pass 为 0，策略无漂移
Effect 证据	python scripts/check_effect_evidence_contract_v32.py	15 项均有最终分类，candidate warning 为 0
冻结	python scripts/check_release_refreeze_v33.py	repeat-five、12 项终止条件、文档和制品均通过
制品整洁度	python scripts/check_release_archive_hygiene_v33.py <archive.zip>	缓存、临时运行目录和疑似凭据为 0

正式制品分为 source、assets、evidence 和 full 四类。轻量 wheel 不携带大型 SPICE 内核；source 包也不携带 assets 和历史 reports。使用前应核对 _SHA256SUMS.txt。